Name: Monvitha

Roll Number:26001014

Branch:CSE (AI&ML)

Section: CSM-3

1.Examination Result and Grace-Mark Validation

2.Attendance Recovery Computation

3.ATM Transaction Sequence Simulation

4.Fuel Consumption and Route Comparison

5.Progressive Water Tariff Analysis

6.Advanced Numeric Pattern Generator

7.Password Policy Validation Engine

8.String-Based Examination Answer Evaluation

9.Student Feedback Sentiment Scoring

10.Text Compression Using Character Runs

PYTHON ACTIVITY-BASED LEARNING

Case Study 1: Examination Result and Grace-Mark Validation

1. Problem Statement

To calculate a student's marks and result by checking pass marks and providing grace marks if eligible.

2. Objectives

- Calculate total and percentage.
- Check pass or fail.
- Apply grace marks if eligible.

3. Assumptions

- Five subjects, each carrying 100 marks.
- Pass mark is 35.
- Maximum 5 grace marks are allowed.
- Grace marks are given if only one subject has 30–34 marks.

4. Algorithm

1. Start.
2. Enter marks in five subjects.
3. Check failed subjects.
4. Apply grace marks if eligible.
5. Calculate total and percentage.
6. Display result.

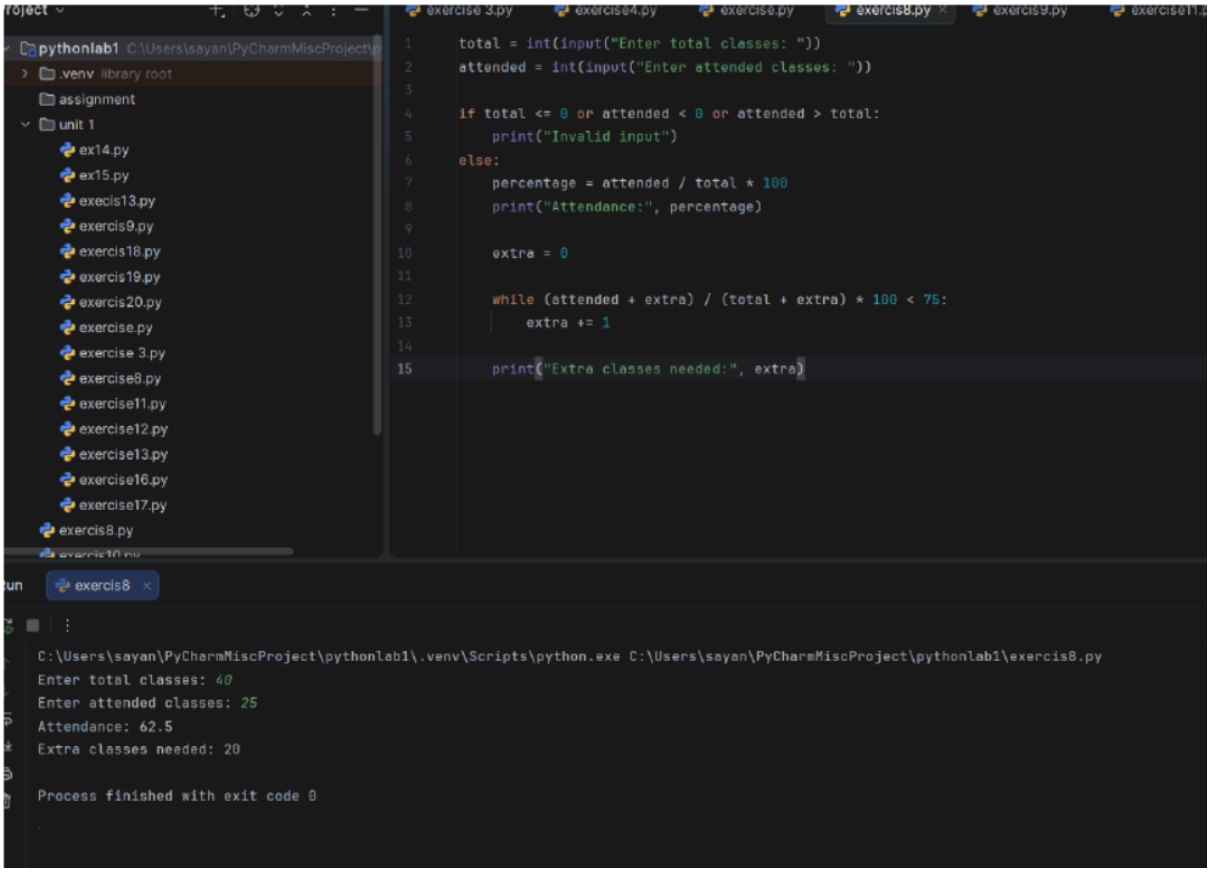
5. Flowchart

Start → Enter marks → Check failed subjects → Apply grace marks → Calculate percentage → Display result → Stop.

6. Pseudocode

```
START
INPUT five subject marks
COUNT failed subjects
IF only one subject failed
   APPLY grace marks if eligible
END IF
CALCULATE total and percentage
DISPLAY result
STOP
```

7. Python Source Code



8. Test Cases

Marks	Expected result
40,50,60,70,80	Pass
32,50,60,70,80	Pass with grace
20,40,50,60,70	Fail

9. Program Input and Output

Input:

Enter marks: 32
Enter marks: 50

Enter marks: 60
Enter marks: 70
Enter marks: 80

Output:

Grace marks awarded
Total: 295
Percentage: 59.0
PASS

10. Conclusion

The program calculates examination results and checks grace-mark eligibility using Python.

Case Study 2: Attendance Recovery Computation

1. Problem Statement

To calculate student attendance percentage and find the additional classes required to reach the target attendance.

2. Objectives

- Calculate attendance percentage.
- Check attendance eligibility.
- Find additional classes required.

3. Assumptions

- Attendance is entered by the user.
- Target attendance is 75%.
- The student attends all future classes.

4. Algorithm

1. Start.
2. Enter total and attended classes.
3. Calculate attendance percentage.
4. Check target percentage.
5. Find additional classes if needed.
6. Display result.

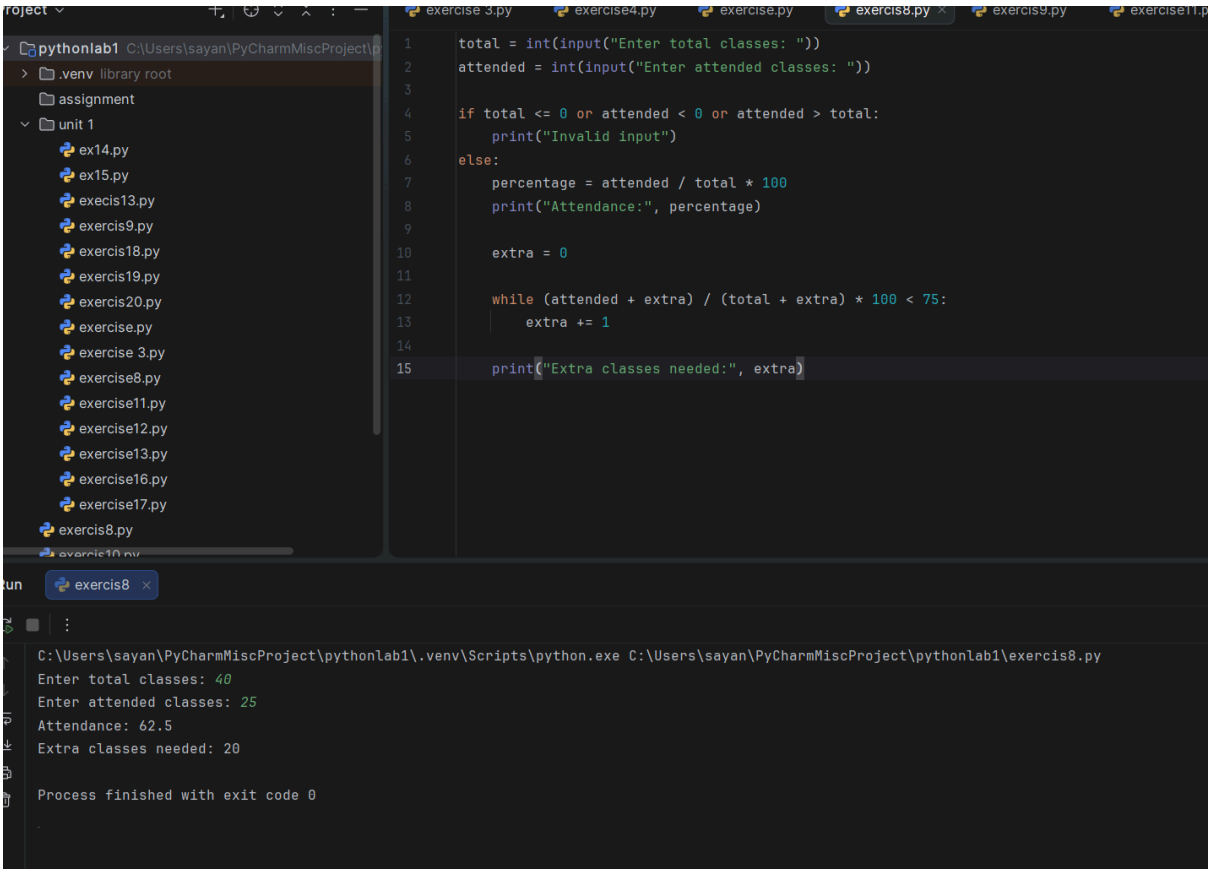
5. Flowchart

Start → Enter attendance → Calculate percentage → Check target → Find extra classes → Display result → Stop.

6. Pseudocode

START
INPUT total and attended classes
CALCULATE percentage
IF percentage >= target
   DISPLAY target achieved
ELSE
   COUNT additional classes using loop
END IF
STOP

7. Python Source Code



8. Test Cases

Total	Attended	Expected result
40	25	20 extra classes
40	35	0 extra classes

50	20	100 extra classes

9. Program Input and Output

Input:

Enter total classes: 40
Enter attended classes: 25

Output:

Attendance: 62.5
Extra classes needed: 20

10. Conclusion

The program calculates attendance percentage and finds the extra classes required using a while loop.

Case Study 3: ATM Transaction Sequence Simulation

1. Problem Statement

To create a simple ATM program that performs balance enquiry, withdrawal, and deposit.

2. Objectives

- Check account balance.
- Withdraw money.
- Deposit money.

3. Assumptions

- Initial balance is entered by the user.
- Withdrawal cannot exceed balance.
- Transaction amount must be positive.

4. Algorithm

1. Start.
2. Enter account balance.
3. Display ATM menu.
4. Select an operation.
5. Perform the transaction.
6. Repeat until Exit.

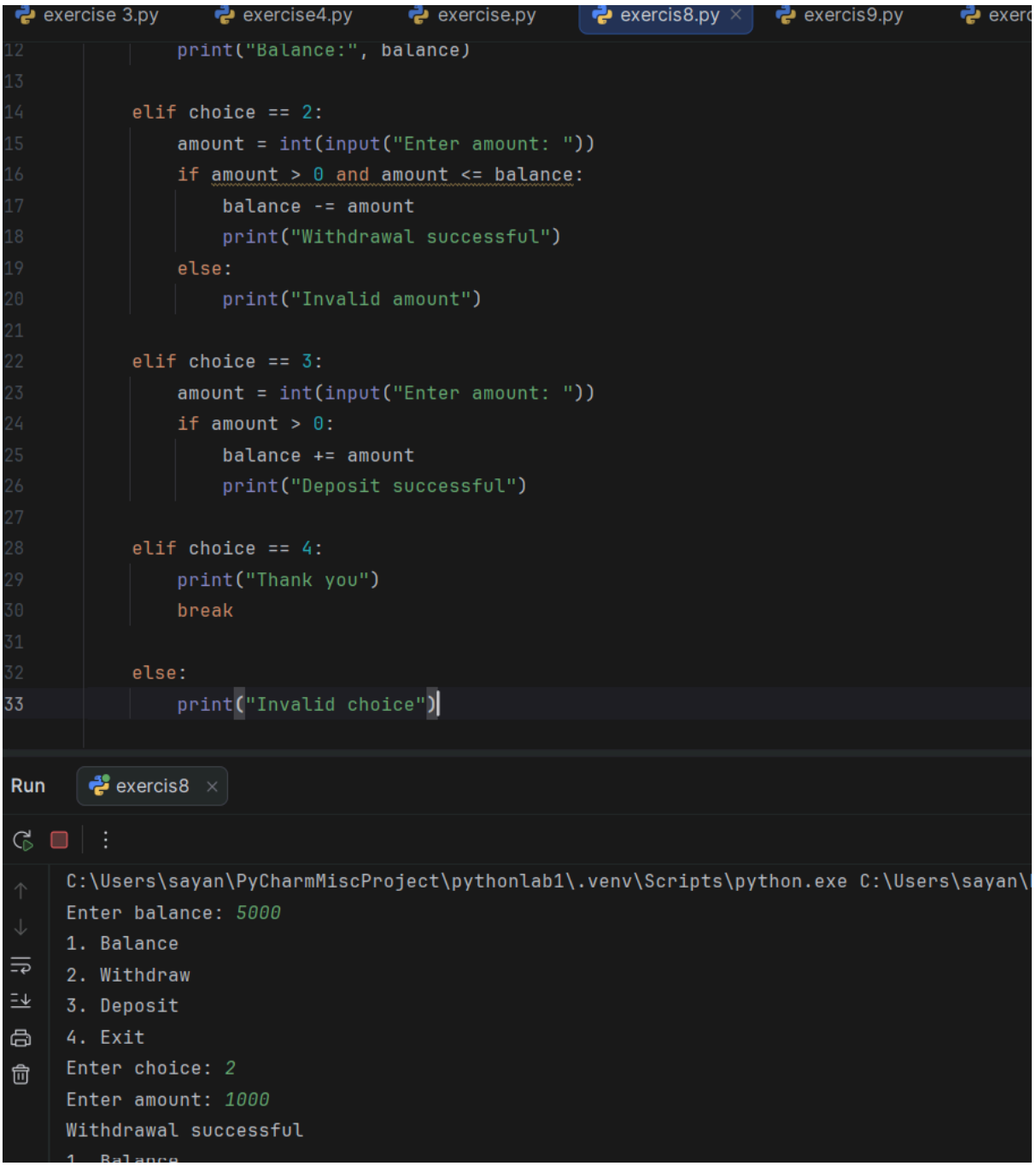
5. Flowchart

Start → Enter balance → Display menu → Select option → Perform transaction → Display balance → Exit or Repeat → Stop.

6. Pseudocode

```
START
INPUT balance
REPEAT
   DISPLAY ATM MENU
   INPUT choice
   PERFORM selected operation
UNTIL choice = 4
STOP
```

7. Python Source Code



8. Test Cases

Operation	Expected result
Balance = 5000	Display 5000

Withdraw 1000 Balance = 4000

Deposit 500 Balance = 4500

9. Program Input and Output

Input:

Enter balance: 5000
Enter choice: 2
Enter amount: 1000

Output:

Withdrawal successful
Balance: 4000

10. Conclusion

The program performs basic ATM operations using loops and conditional statements.

Case Study 4: Fuel Consumption and Route Comparison

1. Problem Statement

To calculate fuel consumption and compare the cost of travelling through two different routes.

2. Objectives

- Calculate fuel consumption.
- Calculate travel cost.
- Compare two routes.

3. Assumptions

- Distance is measured in kilometres.
- Mileage is measured in km/litre.
- Fuel price is the same for both routes.

4. Algorithm

1. Start.
2. Enter two route distances.

3. Enter mileage and fuel price.
4. Calculate fuel required.
5. Calculate route costs.
6. Display the cheaper route.

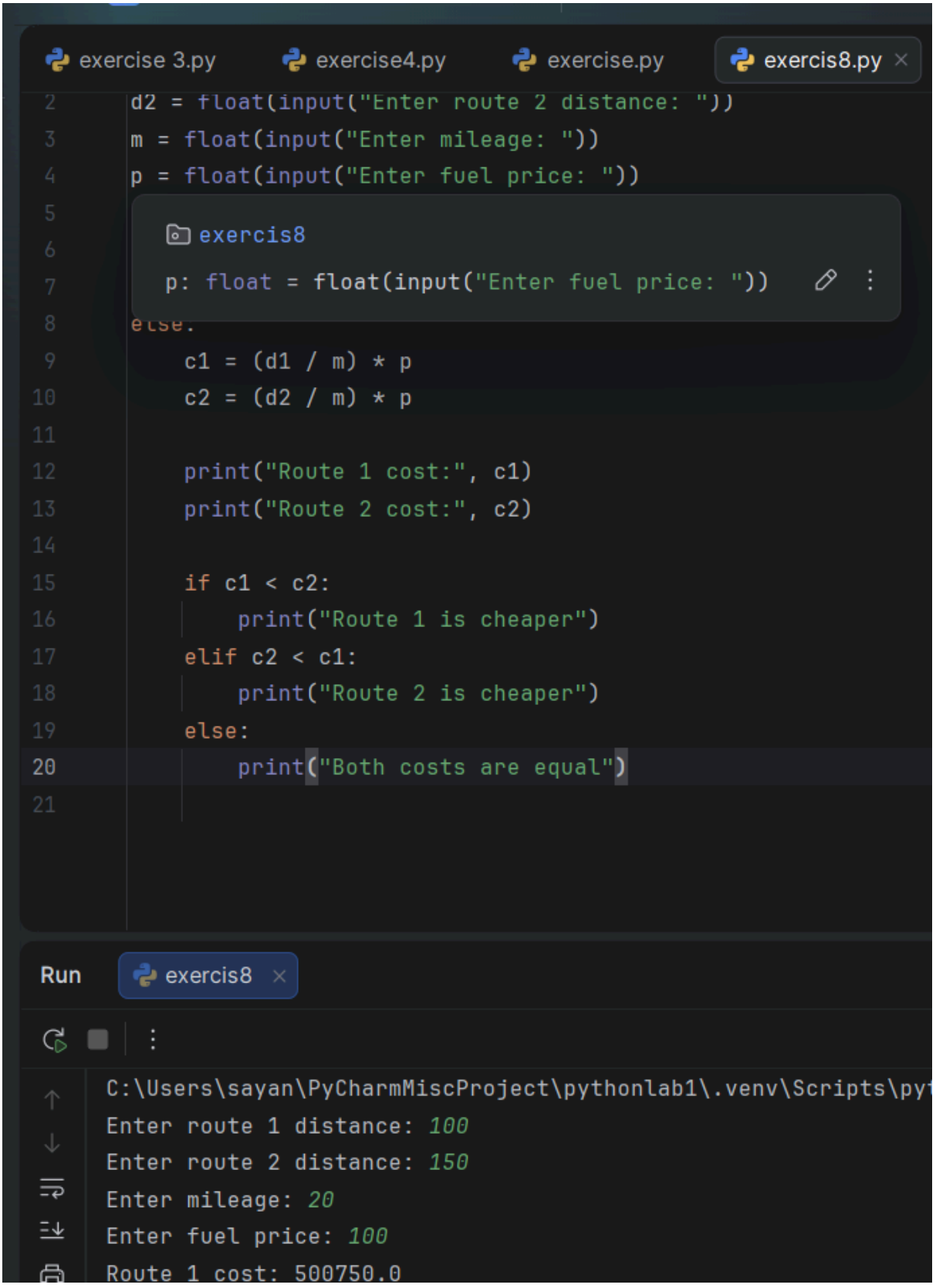
5. Flowchart

Start → Enter details → Calculate fuel → Calculate cost → Compare routes → Display result → Stop.

6. Pseudocode

START
INPUT distances, mileage, fuel price
CALCULATE fuel for both routes
CALCULATE costs
COMPARE costs
DISPLAY cheaper route
STOP

7. Python Source Code


exercise 3.py
exercise4.py
exercise.py
exercis8.py
d2 = float(input("Enter route 2 distance: "))
m = float(input("Enter mileage: "))
p = float(input("Enter fuel price: "))
exercis8
p: float = float(input("Enter fuel price: "))
else:
c1 = (d1 / m) * p
c2 = (d2 / m) * p
print("Route 1 cost:", c1)
print("Route 2 cost:", c2)
if c1 < c2:
print("Route 1 is cheaper")
elif c2 < c1:
print("Route 2 is cheaper")
else:
print("Both costs are equal")
Run
exercis8
C:\Users\sayan\PyCharmMiscProject\pythonlab1\.venv\Scripts\pyt
Enter route 1 distance: 100
Enter route 2 distance: 150
Enter mileage: 20
Enter fuel price: 100
Route 1 cost: 500750.0

8. Test Cases

Route 1	Route 2	Expected result
100 km	150 km	Route 1 cheaper
200 km	100 km	Route 2 cheaper
100 km	100 km	Equal cost

9. Program Input and Output

Input:

Enter route 1 distance: 100
Enter route 2 distance: 150
Enter mileage: 20
Enter fuel price: 100

Output:

Route 1 cost: 500.0
Route 2 cost: 750.0
Route 1 is cheaper

10. Conclusion

The program compares fuel costs of two routes and identifies the economical route.

Case Study 5: Progressive Water Tariff Analysis

1. Problem Statement

To calculate the water bill based on the units consumed using different tariff rates.

2. Objectives

- Calculate water bill.
- Apply different unit rates.
- Display total charges.

3. Assumptions

- First 100 units: ₹2 per unit.
- Next 100 units: ₹3 per unit.
- Above 200 units: ₹5 per unit.

4. Algorithm

1. Start.
2. Enter water units.
3. Check the units.
4. Apply the correct tariff.
5. Calculate total bill.
6. Display bill.

5. Flowchart

Start → Enter units → Check slab → Calculate bill → Display bill → Stop.

6. Pseudocode

```
START
INPUT units
IF units <= 100
   bill = units * 2
ELSE IF units <= 200
   bill = 200 + remaining units * 3
ELSE
   bill = 500 + remaining units * 5
END IF
DISPLAY bill
STOP
```

7. Python Source Code

```python
d2 = float(input("Enter route 2 distance: "))
m = float(input("Enter mileage: "))
p = float(input("Enter fuel price: "))

if d1 <= 0 or d2 <= 0 or m <= 0 or p <= 0:
    print("Invalid input")
else:
    c1 = (d1 / m) * p
    c2 = (d2 / m) * p

    print("Route 1 cost:", c1)
    print("Route 2 cost:", c2)

    if c1 < c2:
        print("Route 1 is cheaper")
    elif c2 < c1:
        print("Route 2 is cheaper")
    else:
        print("Both costs are equal")
```

```
Run  exercis8
C:\Users\sayan\PyCharmMiscProject\pythonlab1\.venv\Scripts\python.exe C:\Users\s
Enter route 1 distance: 100
Enter route 2 distance: 150
Enter mileage: 20
Enter fuel price: 100
Route 1 cost: 500750.0
Route 2 cost: 750.0
Route 2 is cheaper
```

8. Test Cases

Units	Expected bill
80	₹160
150	₹350

250 ₹750

9. Program Input and Output

Input:

Enter water units: 150

Output:

Water bill: 350

10. Conclusion

The program calculates water charges using progressive tariff rates and conditional statements.

Case Study 6: Advanced Numeric Pattern Generator

1. Problem Statement

To generate a number pattern based on the number of rows entered by the user.

2. Objectives

- Generate number patterns.
- Understand nested loops.
- Display numbers in rows.

3. Assumptions

- Number of rows must be between 1 and 20.
- The pattern starts from 1.

4. Algorithm

1. Start.
2. Enter number of rows.
3. Check the input.
4. Use nested loops.
5. Print the number pattern.
6. Stop.

5. Flowchart

Start → Enter rows → Check input → Use nested loops → Print pattern → Stop.

6. Pseudocode

START
INPUT rows
FOR i = 1 TO rows
    FOR j = 1 TO i
        PRINT j
    END FOR
    MOVE TO NEXT LINE
END FOR
STOP

7. Python Source Code

```python
n = int(input("Enter number of rows: "))

if n < 1 or n > 20:
    print("Invalid rows")
else:
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()
```

```
Run  exercis8
C:\Users\sayan\PyCharmMiscProject\pythonlab1\.venv\Scripts\python.exe C:\Users\sayan\PyCharmMiscProject\
Enter number of rows: 4
1
1 2
1 2 3
1 2 3 4

Process finished with exit code 0
```

8. Test Cases

Rows	Expected result

3	Three rows
5	Five rows
0	Invalid rows

9. Program Input and Output

Input:

Enter number of rows: 4

Output:

1
1 2
1 2 3
1 2 3 4

10. Conclusion

The program generates number patterns using nested for loops.

Case Study 7: Password Policy Validation Engine

1. Problem Statement

To check whether a password is strong by checking its length, letters, numbers, and special characters.

2. Objectives

- Check password length.
- Identify uppercase and lowercase letters.
- Check numbers and special characters.

3. Assumptions

- Minimum length is 8.
- Password must contain uppercase, lowercase, digit, and special character.

4. Algorithm

1. Start.

2. Enter password.
3. Check each character.
4. Verify password conditions.
5. Display valid or invalid.
6. Stop.

5. Flowchart

Start → Enter password → Check characters → Verify conditions → Display result → Stop.

6. Pseudocode

```
START
INPUT password
CHECK uppercase, lowercase, digit and special character
IF all conditions are satisfied
   DISPLAY valid password
ELSE
   DISPLAY invalid password
END IF
STOP
```

7. Python Source Code

```python
p = input("Enter password: ")

upper = False
lower = False
digit = False
special = False

for ch in p:
    if ch.isupper():
        upper = True
    elif ch.islower():
        lower = True
    elif ch.isdigit():
        digit = True
    else:
        special = True

if len(p) >= 8 and upper and lower and digit and special:
    print("Valid Password")
else:
    print("Invalid Password")
```

```
un  exercis8 ×

C:\Users\sayan\PyCharmMiscProject\pythonlab1\.venv\Scripts\python.exe C:\Users\sayan\PyCharmMiscProject\python
Enter password: hellow@123
Invalid Password

Process finished with exit code 0
```

8. Test Cases

Password	Expected result
Hello@123	Valid
hello123	Invalid
HELLO@123	Invalid

9. Program Input and Output

Input:

Enter password: Hello@123

Output:

Valid Password

10. Conclusion

The program checks password strength using strings and conditional statements.

Case Study 8: String-Based Examination Answer Evaluation

1. Problem Statement

To compare student answers with the correct answer key and calculate the examination score.

2. Objectives

- Compare answers.
- Count correct answers.
- Calculate marks and percentage.

3. Assumptions

- Each correct answer carries one mark.
- No negative marking.
- Both answer strings must have equal length.

4. Algorithm

1. Start.
2. Enter correct answers.
3. Enter student answers.
4. Compare both strings.
5. Calculate score and percentage.
6. Display result.

5. Flowchart

Start → Enter answers → Compare answers → Calculate score → Display result → Stop.

6. Pseudocode

```
START
INPUT answer key and student answers
IF lengths are equal
   COMPARE each answer
```

COUNT correct answers
CALCULATE percentage
DISPLAY score
ELSE
DISPLAY invalid answer count
END IF
STOP

7. Python Source Code

```
key = input("Enter answer key: ").upper()
ans = input("Enter student answers: ").upper()

if len(key) == 0:
    print("Empty answer key")
elif len(key) != len(ans):
    print("Invalid answers")
else:
    score = 0

    for i in range(len(key)):
        if key[i] == ans[i]:
            score += 1

    percentage = score / len(key) * 100

    print("Score:", score)
    print("Percentage:", percentage)
```

```
un  exercis8 ×

C:\Users\sayan\PyCharmMiscProject\pythonlab1\.venv\Scripts\python.exe C:\Users\sayan\PyCh
Enter answer key: ABCD
Enter student answers: ABBD
Score: 3
Percentage: 75.0

Process finished with exit code 0
```

8. Test Cases

Answer key	Student answers	Result
ABCD	ABCD	4 marks
ABCD	ABBD	3 marks
ABCD	ABC	Invalid

9. Program Input and Output

Input:

Enter answer key: ABCD
Enter student answers: ABBD

Output:

Score: 3
Percentage: 75.0

10. Conclusion

The program evaluates student answers and calculates marks using string comparison.

Case Study 9: Student Feedback Sentiment Scoring

1. Problem Statement

To analyse student feedback by identifying positive and negative words and displaying the overall sentiment.

2. Objectives

- Accept student feedback.
- Count positive and negative words.
- Identify the sentiment.

3. Assumptions

- Positive words: good, excellent, nice, helpful.

- Negative words: bad, poor, worst, boring.
- Equal counts indicate neutral feedback.

4. Algorithm

1. Start.
2. Enter feedback.
3. Convert text to lowercase.
4. Count positive and negative words.
5. Compare the counts.
6. Display sentiment.

5. Flowchart

Start → Enter feedback → Count positive and negative words → Compare counts → Display sentiment → Stop.

6. Pseudocode

```
START
INPUT feedback
COUNT positive and negative words
IF positive > negative
   DISPLAY positive
ELSE IF negative > positive
   DISPLAY negative
ELSE
   DISPLAY neutral
END IF
STOP
```

7. Python Source Code

```python
text = input("Enter feedback: ").lower()

positive = ["good", "excellent", "nice", "helpful"]
negative = ["bad", "poor", "worst", "boring"]

p = 0
n = 0

words = text.split()

for word in words:
    if word in positive:
        p += 1
    elif word in negative:
        n += 1

if p > n:
    print("Positive Feedback")
elif n > p:
    print("Negative Feedback")
else:
    print("Neutral Feedback")
```

exercis8

```
C:\Users\sayan\PyCharmMiscProject\pythonlab1\.venv\Scripts\python.exe C:\Users\sayan\PyCh
Enter feedback: Teaching is good and helpful
Positive Feedback

Process finished with exit code 0
```

8. Test Cases

Feedback	Expected result
Good and excellent	Positive

Bad and poor	Negative
Good and bad	Neutral

9. Program Input and Output

Input:

Enter feedback: Teaching is good and helpful

Output:

Positive Feedback

10. Conclusion

The program analyses feedback using string operations and identifies positive, negative, or neutral sentiment.

Case Study 10: Text Compression Using Character Runs

1. Problem Statement

To compress text by counting consecutive repeated characters and displaying each character with its count.

2. Objectives

- Accept a text string.
- Count repeated characters.
- Generate compressed text.

3. Assumptions

- Consecutive repeated characters are counted.
- Character comparison is case-sensitive.
- Each character is followed by its count.

4. Algorithm

1. Start.
2. Enter a text string.
3. Count repeated characters.
4. Store character and count.

5. Display compressed text.
6. Stop.

5. Flowchart

Start → Enter text → Count repeated characters → Create compressed text → Display result → Stop.

6. Pseudocode

```
START
INPUT text
SET count = 1
CHECK each character
IF next character is same
   INCREASE count
ELSE
   STORE character and count
   RESET count
END IF
DISPLAY compressed text
STOP
```

7. Python Source Code

```python
text = input("Enter text: ")

if text == "":
    print("Empty input")
else:
    result = ""
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            result += text[i - 1] + str(count)
            count = 1

    result += text[-1] + str(count)

    print("Compressed text:", result)
```

```
exercis8 ×

C:\Users\sayan\PyCharmMiscProject\pythonlab1\.venv\Scripts\python.exe C:
Enter text: aaabbc
Compressed text: a3b2c1

Process finished with exit code 0
```

8. Test Cases

Input	Expected output

aaabbc	a3b2c1
hello	h1e1l2o1
xxxxzz	x4z2

9. Program Input and Output

Input:

Enter text: aaabbc

Output:

Compressed text: a3b2c1

10. Conclusion

The program compresses text by counting repeated characters using loops and string operations.